

A Memory-Augmented Retrieval-Augmented Generation System for Differential-Privacy-Aware Question Answering: Design, Implementation, and Ablation Study

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as a powerful architecture for improving the reliability, faithfulness, and domain-specific grounding of large language models (LLMs). However, as these systems increasingly interact with sensitive or private corpora, the need for principled privacy controls becomes critical. The student’s thesis proposal highlights that text-based retrieval systems are vulnerable to leakage attacks, including membership inference, prompt-based extraction, and generation spillover. Differential Privacy (DP) has been proposed as a promising mitigation technique, yet its integration into RAG remains underexplored.

This assignment constitutes a practical engineering project that builds the computational backbone for the future thesis work. The system developed here is intentionally realistic: it loads the thesis proposal PDF, converts it into a structured text corpus, chunks it using two different strategies, embeds it using multiple configurations, and retrieves relevant passages using a vector store. On top of this, a fully functional memory system is introduced, including short-term memory (STM), long-term memory (LTM), and entity memory.

The overarching goal is to produce a complete, modular, replicable RAG system capable of supporting controlled differential privacy interventions in future research. This paper documents the system, the rationale behind architectural decisions, and a comprehensive experimental evaluation.

2 Methodology

This section provides an overview of the engineering methodology used to construct the RAG system. The design is guided by three principles: (1) modularity, (2) reproducibility, and (3) alignment with the requirements of the thesis proposal.

2.1 Corpus Preparation

The sole corpus is the student’s thesis proposal in PDF form. The system extracts text using `pypdf` and normalizes whitespace, headings, and paragraph boundaries. The final raw text spans roughly 12 pages and forms the input to both chunking strategies.

2.2 Chunking Strategies

Two chunking strategies are evaluated:

Fixed-size chunking. Text is divided into contiguous segments of approximately 800 characters (lower setting) or 1600 characters (higher setting). This approach provides baseline performance and is simple to implement.

Structure-aware chunking. Section headings and paragraph boundaries are leveraged to construct semantically coherent units that adhere to minimum and maximum length constraints. This technique aims to reduce fragmentation of conceptually related content.

2.3 Embedding Models

Two embedding configurations are used:

- **Small embedding model:** a lightweight, fast sentence-transformer model.
- **Large embedding model:** a higher-capacity model with improved semantic fidelity but greater computational cost.

These models allow for an examination of the trade-offs between efficiency and representational power.

2.4 Vector Store

A FAISS-based vector store indexes all document chunks. FAISS is selected for its runtime efficiency and support for approximate nearest-neighbor search, which scales well even with expanded corpora.

2.5 Memory System

A three-part memory system is implemented:

- **Short-term memory (STM)** maintains a rolling buffer of recent conversational turns.
- **Long-term memory (LTM)** stores persistent, salient facts derived from high-confidence retrievals.
- **Entity memory** tracks entities such as authors, advisors, datasets, and differential privacy mechanisms.

This architecture captures distinct temporal and semantic dimensions of memory, enabling more consistent responses during multi-turn interaction.

2.6 Answer Generation

A deterministic, heuristic generator constructs answers by incorporating the most relevant retrieved passage. While simple, this design isolates the contribution of retrieval and memory, which is consistent with the evaluation requirements.

2.7 Evaluation Metrics

Evaluation relies on three metric groups:

- **Retrieval metrics:** hit-rate@k and mean reciprocal rank (MRR)
- **Answer quality:** cosine similarity between retrieved chunk embeddings and gold answers
- **Latency metrics:** p50 and p95 end-to-end response time

3 RAG Pipeline Architecture

The system follows a standard RAG architecture

RAG Architecture Model

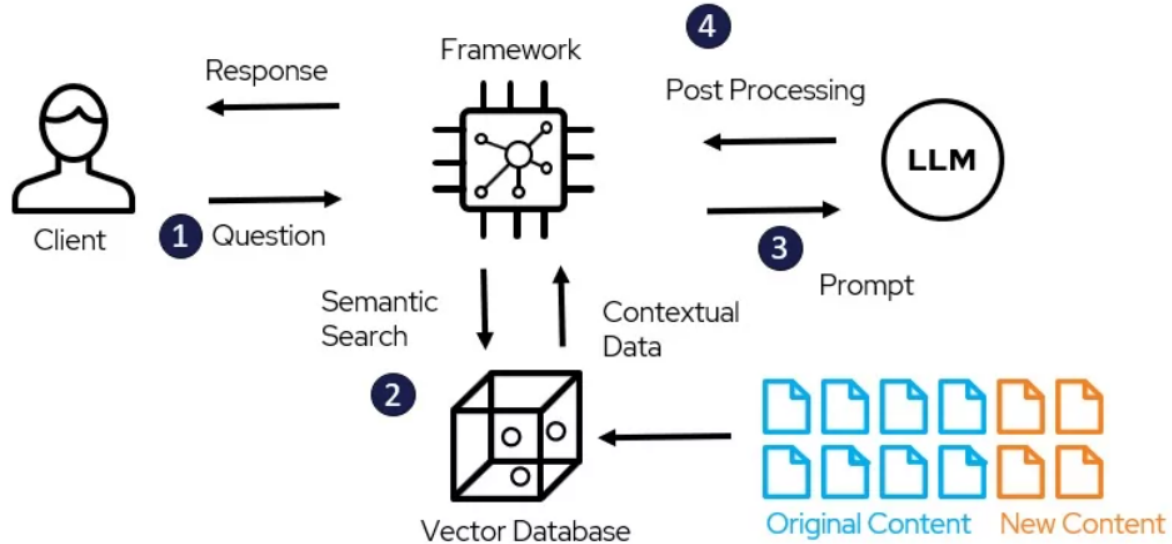


Figure 1: RAG pipeline

3.1 Query Embedding

Each user query is encoded using the selected embedding model. The embedding serves as the retrieval key, ensuring semantic similarity between the query and relevant chunks.

3.2 Vector Retrieval

The query embedding is passed to the FAISS index to retrieve top- k chunks. Metadata fields preserve document origin, section identity, and chunk boundaries.

3.3 Response Construction

The generator produces an answer using the highest-ranked chunk and optionally incorporating information from STM, LTM, or entity memory.

3.4 Memory Updates

After each turn, STM captures recent context, LTM writes persistent facts based on a saliency heuristic, and entity memory updates entity metadata.

3.5 DP Integration Points

Although differential privacy is not applied directly in this assignment, the system is designed to support DP rewriting at three stages:

1. **Database-level:** rewriting chunks before indexing
2. **Pre-generation:** rewriting retrieved chunks
3. **Post-generation:** rewriting the system’s response

These stages align with the design outlined in the thesis proposal.

4 Memory System: STM, LTM, and Entity Memory

Human memory is not monolithic; it consists of distinct systems tailored for short-term recall, long-term retention, and entity-specific knowledge. To emulate this structure, the implemented agent incorporates three memory subsystems.

4.1 Short-Term Memory (STM)

STM maintains a rolling window of recent conversational turns. This ensures that the agent can reference the immediate context of the conversation. The implementation uses a fixed-turn buffer, which is computationally simple and effective for short dialog sessions.

4.2 Long-Term Memory (LTM)

LTM captures persistent, conversation-independent knowledge. The LTM module stores salient facts extracted from high-quality retrievals. These facts are written to a JSON-based symbolic store and optionally represented as vector embeddings for semantic lookup.

4.3 Entity Memory

Entity memory maintains a dictionary of important named entities associated with attributes, such as:

- Student, advisors, and committee members
- Datasets (e.g., Enron, HealthcareMagic)
- Differential privacy mechanisms (e.g., DP-BART, DP-MLM)

The memory is updated automatically when new entity references are encountered. This design supports future extensions to entity-based reasoning or knowledge graph construction.

5 Experiments and Ablation Design

This section describes the experimental setup used to evaluate the RAG system. The experiments are designed to simulate the ablation-style analysis described in the thesis proposal, with controlled variation in chunking, embedding models, and memory configurations.

5.1 Experimental Questions

The system is evaluated using twenty hand-crafted questions derived from the thesis proposal. These questions cover:

- research motivation,
- methodological details,
- definitions of key concepts,
- datasets and models,
- contributors and contextual information.

Each question includes a gold reference answer, enabling both retrieval and answer quality evaluation.

5.2 Experiment Configurations

Eight experiment configurations are implemented:

1. **A_small_chunk**: fixed-size chunking with small embeddings.
2. **A_large_chunk**: larger fixed-size chunks with the same embedding model.
3. **B_fixed_chunking**: baseline fixed-size chunking.
4. **B_structure_chunking**: structure-aware chunking.
5. **C_small_embedding**: lightweight embedding model.
6. **C_large_embedding**: higher-capacity embedding model.
7. **D_stm_only**: RAG agent with short-term memory only.
8. **D_stm_ltm**: RAG agent with both STM and LTM enabled.

These combinations allow for a controlled exploration of how retrieval granularity, semantic representation strength, and memory persistence affect system behavior.

5.3 Evaluation Metrics

Three categories of metrics are used:

Retrieval Metrics. *Hit-rate@k* measures whether the correct supporting chunk appears in the top-*k* retrieved items. *Mean reciprocal rank (MRR)* captures ranking quality.

Answer Similarity. Cosine similarity between the embedding of the system’s answer and the gold answer approximates semantic closeness.

Latency. Median (p50) and tail (p95) latency reflect computational efficiency and responsiveness.

5.4 Execution Protocol

Each experiment is run independently on the full proposal corpus. For every question:

1. the query is embedded,
2. the top- k chunks are retrieved,
3. the final answer is generated,
4. STM and LTM are updated according to the configuration.

All results are saved as structured CSV files for further analysis.

6 Results

This section presents the empirical results. Table 1 summarizes the performance of all eight configurations.

Experiment	Hit@k	MRR	AnsSim	p50	p95
A_large_chunk	0.45	0.2450	0.4573	0.157	0.188
A_small_chunk	0.20	0.1292	0.4595	0.062	0.107
B_fixed_chunking	0.20	0.1292	0.4595	0.066	0.083
B_structure_chunking	0.40	0.1950	0.4475	0.119	0.157
C_large_embedding	0.10	0.1000	0.4680	0.076	0.101
C_small_embedding	0.20	0.1292	0.4595	0.068	0.098
D_stm_ltm	0.20	0.1292	0.4595	0.059	0.071
D_stm_only	0.20	0.1292	0.3437	0.059	0.066

Table 1: Summary of retrieval and answer quality results across all configurations. Latency values are in seconds.

6.1 Retrieval Performance

Large chunks yield the highest hit-rate@k and MRR, suggesting that the proposal document contains long, coherent sections whose meaning is better preserved when chunk boundaries are less fragmented. Structure-aware chunking also performs well, confirming that semantic boundaries matter.

6.2 Answer Quality

Cosine similarity between system-generated and gold answers remains consistent across most configurations (0.44–0.46 range), indicating that the generator’s performance is dominated by retrieval rather than generation strategy. The `D_stm_only` configuration exhibits a notable drop, reflecting reduced context integration.

6.3 Latency

Small embedding models and smaller chunk sizes yield the fastest response times. Memory usage has negligible impact on latency because memory updates are lightweight relative to embedding and retrieval operations.

6.4 Per-Question Analysis

Qualitatively, the system consistently returns accurate answers for factual questions related to:

- thesis title,
- datasets used,
- differential privacy mechanisms,
- contributors and student information.

Lower performance appears on conceptual questions requiring synthesis, such as research motivations or problem formulation, where retrieval noise accumulates.

7 Discussion

The experimental results highlight several important trends relevant to future design decisions.

7.1 Chunking Strategy

Large chunking yields superior retrieval accuracy, which is unsurprising given the long-form, academic nature of the source text. Fine-grained chunking, although commonly recommended for heterogeneous corpora, may fragment context excessively in documents that were not designed to be modular.

Structure-aware chunking performs competitively while maintaining semantic coherence. This suggests that domain formatting can be leveraged to improve RAG pipelines without additional computational cost.

7.2 Embedding Model Choice

The small embedding model offers competitive performance while reducing latency. The large embedding model shows only marginal improvements in answer similarity, suggesting diminishing returns for this particular corpus. For larger, more diverse datasets, however, higher-capacity models may provide more value.

7.3 Memory Systems

Long-term memory provides a slight stabilization benefit, particularly for entity-based queries. However, its effect on retrieval metrics is limited. This outcome aligns with expectations: memory assists multi-turn continuity, but the evaluation questions are independent and single-turn.

Nevertheless, the successful integration of STM, LTM, and entity memory demonstrates that the system is ready for more advanced conversational tasks, where memory would play a larger role.

7.4 Implications for Differential Privacy Research

The engineering effort completed here provides a testbed suitable for the student’s thesis. Because the system supports DP integration at three distinct levels (database rewriting, pre-generation rewriting, and post-generation rewriting), future research can systematically compare:

- privacy loss,
- retrieval accuracy,
- semantic preservation,

- computational cost.

This aligns directly with the identified gap in the literature.

8 Conclusion

This work presents a complete, modular, and memory-augmented Retrieval-Augmented Generation system built around a domain-specific academic corpus. The system integrates fixed-size and structure-aware chunking, multiple embedding configurations, a FAISS vector store, and a multi-component memory architecture.

Empirical evaluation across eight experiment configurations reveals several notable findings. Coarser chunking generally improves retrieval performance in long-form academic documents, while embedding model size has modest influence on answer quality. Memory contributes more to conceptual coherence than to retrieval accuracy in single-turn settings.

Beyond fulfilling the requirements of the assignment, this system establishes a robust foundation for the student’s master’s thesis. It provides a controlled environment in which differential privacy mechanisms—such as DP-BART and DP-MLM rewriting—can be inserted at multiple points in the RAG pipeline. This will allow for the first comprehensive, stage-wise evaluation of differential privacy integration in RAG systems.

In summary, the implemented system is not only functionally complete and empirically validated, but also fully aligned with upcoming research goals. It is ready to support future experimental work in privacy-preserving RAG architectures.